

[Course Home \(/learn/hadoop/home/welcome\)](#) > [Week 4 \(/learn/hadoop/home/week/4\)](#) > [Lesson 2: Map/Reduce Examples and Principles](#)

Assignment: Joining Data

You have not submitted. You must earn 80/100 points to pass.

Deadline Pass this assignment by November 15, 11:59 PM PT.

Instructions (</learn/hadoop/programming/6ogWY/joining-data>)

My submission (</learn/hadoop/programming/6ogWY/joining-data/submission>)

Discussions (</learn/hadoop/programming/6ogWY/joining-data/discussions>)

Introduction to Map/Reduce Module, Programming Assignment, Lesson 2

Exercise in Joining data with streaming using Python code

In Lesson 2 of the Introduction to Map/Reduce module the Join task was described. In this assignment, first you are given a Python mapper and reducer to perform the Join described in the video (3rd video of lesson). The purpose of the first assignment is mostly to provide an example for the second one. Your second assignment will be to modify that Python code (or if you are so inclined write one from scratch) to perform a different Join. You will be asked to upload an output file(s).

1. Follow the steps from the Wordcount assignment to set up the following files on Cloudera: join1_mapper.py join1_reducer.py join1_FileA.txt join1_FileB.txt (see below)

Don't forget to enter the following to make it executable

```
> chmod +x join1_mapper.py > chmod +x join1_reducer.py
```

Help Center

2. Follow the steps from the Wordcount assignment to set up the data in HDFS

3. Test the program in serial execution using the following Unix utilities and piping commands:

('cat' prints out the text files standard output; '|' pipes the standard output to the standard input of the join_mapper program, etc..)

```
> cat join1_File*.txt | join1_mapper.py | sort | join1_reducer.py
```

To debug programs in serial execution one should use small datasets and possibly extra print statements in the program. Debugging with map/reduce is harder but hopefully not necessary for this assignment, but see the `--debug` option in the streaming command, which will help view standard input/output/error from the map/reduce functions.

4. Run the Hadoop streaming command: (Note that your file paths may differ. Note the '\ ' just means the command continues on next line.)

```
> hadoop jar /usr/lib/hadoop-mapreduce/hadoop-streaming.jar \
  -input /user/cloudera/input \
  -output /user/cloudera/output_join \
  -mapper /home/cloudera/join1_mapper.py \
  -reducer /home/cloudera/join1_reducer.py
```

5. A new join problem:

a) First generate some datasets using the scripts (see below) as follows:

```
> sh make_data_join2.txt
```

(this is a script that produces 6 files:

```
python make_join2data.py y 1000 13 > join2_gennumA.txt
```

```
python make_join2data.py y 2000 17 > join2_gennumB.txt
```

...)

Use HDFS commands to copy all 6 files into one HDFS directory, just like step2 above and in the wordcount assignment.

Note: These datasets are pseudo-randomly generated so all output is the same for any environment. The file are not large but big enough to make solving the assignment by hand time consuming. One could put the data in a database but that would defeat the assignment purpose!

b) The datasets generated in a) have the following information:

join2_gennum*.txt consist of: <TV show, count>

Description: A TV show title (or maybe its a TV show genre) and a count of how many viewers saw that show, for example:

Almost_News, 25

Hourly_Show, 30

Hot_Cooking, 7

Almost_News, 35

Postmodern_Family, 8

Baked_News, 15

Dumb_Games, 60

...

join2_genchan*.txt consists of: <TV show title, channel>

Description: A TV show title and the channel it was shown on, for example:

Almost_News, ABC

Hourly_Show, COM

Hot_Cooking, FNT

Postmodern_Family, NBC

Baked_News, FNT

Dumb_Games, ABC

...

c) Your Task: Implement the following join request in Map/Reduce:

What is the total number of viewers for shows on ABC?

(In pseudo-SQL it might be something like: Select sum(viewer count) from File A, File B where FileA.TV show = FileB.TV show and FileB.Channel='ABC' grouped by TV show)

c) Upload the resulting output from the reducers, use numReduceTasks=1

Output: <TVshow title, total viewers of that show>

For example, from the above file snippets the output would be:

Almost_News 50

Dumb_Games 60

...

d) Data Notes:

1 TV show titles have no blank;

2 Channels have 3 letters

3 TV show titles appear multiple times, with different counts

4 TV show and channel might also appear multiple times

5 TV show could appear on multiple channels

6 The output should have no commas or punctuation, only 1 blank between title and number

e) Programming Notes and Suggestions:

You can choose to use 2 map/reduce jobs, but you can do it in one easily. If you are not a Python programmer then you should review the join1 code, and wordcount code. In fact, you should consider starting with the join1_mapper and join1_reducer. I believe all the logic and function examples you will need are in the wordcount and join1 code

Hint 1: The new join mapper is like join1_mapper but instead of stripping dates from the key field it should be selecting rows related to 'ABC'.

Hint 2: The new join reducer is like join1_reducer but instead of building up a list of dates & counts, it should be summing viewer counts to keep a running total. Make sure you understand what will be in the intermediate output files that become reducer input (after the mapper and after Hadoop shuffle & group). Look carefully at the groups that are present in the reducer input to see what conditions your reducer will have to check for. You can test the mapper output using the Unix piping mentioned above.

Hint 3: This new join task has some overlap with wordcounting task, and you might also look the wordcount code to review the counting and updating logic, as well as functions that handle strings and integers.

d) The code and text files below are

join1_mapper.py

join1_reducer.py

join1_FileA.txt

join1_FileB.txt

make_join2data.py

make_data_join2.txt (a short command line script)

```
#!/usr/bin/env python
import sys

# -----
# This mapper code will input a <date word, value> input file, and move date into
# the value field for output
#
# Note, this program is written in a simple style and does not full advantage of Python
# data structures, but I believe it is more readable
#
# Note, there is NO error checking of the input, it is assumed to be correct
# meaning no extra spaces, missing inputs or counts, etc..
#
# See # see https://docs.python.org/2/tutorial/index.html for details and python tutorials
#
# -----

for line in sys.stdin:
    line = line.strip() #strip out carriage return
    key_value = line.split(",") #split line, into key and value, returns a list
    key_in = key_value[0].split(" ") #key is first item in list
    value_in = key_value[1] #value is 2nd item

    #print key_in
    if len(key_in)>=2: #if this entry has <date word> in key
        date = key_in[0] #now get date from key field
        word = key_in[1]
        value_out = date+" "+value_in #concatenate date, blank, and value_in
        print( '%s\t%s' % (word, value_out) ) #print a string, tab, and string
    else: #key is only <word> so just pass it through
        print( '%s\t%s' % (key_in[0], value_in) ) #print a string tab and string

#Note that Hadoop expects a tab to separate key value
#but this program assumes the input file has a ',' separating key value
```

```
#!/usr/bin/env python
import sys

# -----
# This reducer code will input a <word, value> input file, and join words together
# Note the input will come as a group of lines with same word (ie the key)
# As it reads words it will hold on to the value field
#
# It will keep track of current word and previous word, if word changes
# then it will perform the 'join' on the set of held values by merely printing out
# the word and values. In other words, there is no need to explicitly match keys b/c
# Hadoop has already put them sequentially in the input
#
# At the end it will perform the last join
#
#
# Note, there is NO error checking of the input, it is assumed to be correct, meaning
# it has word with correct and matching entries, no extra spaces, etc.
#
# see https://docs.python.org/2/tutorial/index.html for python tutorials
#
# San Diego Supercomputer Center copyright
# -----

prev_word      = " "           #initialize previous word to blank string
months         = ['Jan', 'Feb', 'Mar', 'Apr', 'Jun', 'Jul', 'Aug', 'Sep', 'Nov', 'Dec']

dates_to_output = [] #an empty list to hold dates for a given word
day_cnts_to_output = [] #an empty list of day counts for a given word
# see https://docs.python.org/2/tutorial/datastructures.html for list details

line_cnt      = 0 #count input lines

for line in sys.stdin:
    line      = line.strip()      #strip out carriage return
    key_value = line.split('\t')  #split line, into key and value, returns a list
    line_cnt  = line_cnt+1

    #note: for simple debugging use print statements, ie:
    curr_word = key_value[0]      #key is first item in list, indexed by 0
    value_in  = key_value[1]      #value is 2nd item

    #-----
    # Check if its a new word and not the first line
    # (b/c for the first line the previous word is not applicable)
    # if so then print out list of dates and counts
    #-----
    if curr_word != prev_word:

        # -----
        #now write out the join result, but not for the first line input
        # -----
        if line_cnt>1:
            for i in range(len(dates_to_output)): #loop thru dates, indexes start at 0
                print('{0} {1} {2} {3}'.format(dates_to_output[i], prev_word, day_cnts_to_output[i], curr_word_total_cnt))
            #now reset lists
            dates_to_output = []
            day_cnts_to_output = []
            prev_word      = curr_word #set up previous word for the next set of input lines

    # -----
    #whether or not the join result was written out,
    # now process the curr word

    #determine if its from file <word, total-count> or < word, date day-count>
    # and build up list of dates, day counts, and the 1 total count
    # -----
    if (value_in[0:3] in months):

        date_day = value_in.split() #split the value field into a date and day-cnt

        #add date to lists of the value fields we are building
        dates_to_output.append(date_day[0])
        day_cnts_to_output.append(date_day[1])
    else:
        curr_word_total_cnt = value_in #if the value field was just the total count then its
                                       #the first (and only) item in this list

    # -----
    #now write out the LAST join result
    # -----
    for i in range(len(dates_to_output)): #loop thru dates, indexes start at 0
        print('{0} {1} {2} {3}'.format(dates_to_output[i], prev_word, day_cnts_to_output[i], curr_word_total_cnt))
```

```
able,991
about,11
burger,15
actor,22
```

```
Jan-01 able,5
Feb-02 about,3
Mar-03 about,8
Apr-04 able,13
Feb-22 actor,3
Feb-23 burger,5
Mar-08 burger,2
Dec-15 able,100
```

```
#!/usr/bin/env python
import sys

# -----
# (make_join2data.py) Generate a random combination of titles and viewer counts, or channels
# this is a simple version of a congruential generator,
# not a great random generator but enough
# -----

chans = ['ABC', 'DEF', 'CNO', 'NOX', 'YES', 'CAB', 'BAT', 'MAN', 'ZOO', 'XYZ', 'BOB']
sh1 = ['Hot', 'Almost', 'Hourly', 'PostModern', 'Baked', 'Dumb', 'Cold', 'Surreal', 'Loud']
sh2 = ['News', 'Show', 'Cooking', 'Sports', 'Games', 'Talking', 'Talking']
vwr = range(17,1053)

chvnm=sys.argv[1] #get number argument, if its n, do numbers not channels,

lch=len(chans)
lsh1=len(sh1)
lsh2=len(sh2)
lvwr=len(vwr)
ci=1
s1=2
s2=3
vwi=4
ri=int(sys.argv[3])
for i in range(0,int(sys.argv[2])): #arg 2 is the number of lines to output

    if chvnm=='n': #no numuber
        print('{0}_{1},{2}'.format(sh1[s1],sh2[s2],chans[ci]))
    else:
        print('{0}_{1},{2}'.format(sh1[s1],sh2[s2],vwr[vwi]))
    ci=(5*ci+ri) % lch
    s1=(4*s1+ri) % lsh1
    s2=(3*s1+ri+i) % lsh2
    vwi=(2*vwi+ri+i) % lvwr

    if (vwi==4): vwi=5
```

```
python make_join2data.py y 1000 13 > join2_gennumA.txt
python make_join2data.py y 2000 17 > join2_gennumB.txt
python make_join2data.py y 3000 19 > join2_gennumC.txt
python make_join2data.py n 100 23 > join2_genchanA.txt
python make_join2data.py n 200 19 > join2_genchanB.txt
python make_join2data.py n 300 37 > join2_genchanC.txt
```

How to submit

When you're ready to submit, you can upload files for each part of the assignment on the "My submission" tab.

 (/learn/hadoop/supplement/9Jb47/lesson-2-mapreduce-

examples-and-principles-slides) (https://accounts.coursera.org/learn/hadoop-programming-6ogWY/joining-data?turn_to=https://learner.coursera.help/hc)

 (/learn/hadoop/lecture/9cq0R/introduction-to-apache-

spark)